DNA Sequence Alignment: Implementation & Performance Analysis of Needleman–Wunsch Dynamic Programming and Hirschberg's Memory-Efficient Divide-and-Conquer Algorithm

This project implements and compares two algorithms for the global DNA sequence alignment problem: the classical Needleman–Wunsch dynamic programming (DP) algorithm and Hirschberg's linear-space divide-and-conquer algorithm. Both algorithms always find an alignment of the same optimal cost under a fixed gap penalty and mismatch cost matrix, but they differ sharply in memory use: the basic DP algorithm needs $O(mn)$ space, while Hirschberg's algorithm reduces this to $O(m+n)$ at the cost of a modest constant-factor slowdown in running time.

1 Problem Statement

Given two DNA strings X and Y over the alphabet $\{A, C, G, T\}$, the *global sequence alignment* problem asks for the lowest-cost way to align X and Y by inserting gap characters ('_') into either string so that the resulting strings have equal length. Each aligned column incurs a cost: a **gap penalty** δ if one character in the column is a gap, or a **mismatch/match cost** $\alpha(x, y)$ otherwise, taken from a fixed substitution cost matrix. The goal is to minimize the total cost summed over all columns.

2 Scoring Scheme

Gap penalty: $\delta = 30$

Mismatch cost matrix $\alpha(x, y)$:

	A	C	G	T
A	0	110	48	94
C	110	0	118	48
G	48	118	0	110
T	94	48	110	0

3 Input Format

Rather than storing raw DNA strings, each input file stores a compressed description of each sequence: an initial seed string over $\{A, C, G, T\}$, plus a list of 0-indexed insertion positions. The full sequence is reconstructed by repeatedly duplicating the current string and splicing the copy in at the given position:

$$\texttt{result} \leftarrow \texttt{result}[0 : \texttt{pos} + 1] \; + \; \texttt{result} \; + \; \texttt{result}[\texttt{pos} + 1 :]$$

This roughly doubles the string length at each step, so long test sequences can be specified compactly. Both X and Y are encoded this way, one after the other, in a single input file.

4 Algorithms

4.1 Basic Dynamic Programming (`basic.py`)

The standard Needleman–Wunsch recurrence builds a full $(m{+}1) \times (n{+}1)$ cost table `OPT`, where $m = |X|$ and $n = |Y|$:

$$OPT[i][j] = \begin{cases} i \cdot \delta & j = 0 \\ j \cdot \delta & i = 0 \\ \min\big(OPT[i-1][j-1] + \alpha(X_i, Y_j),\ OPT[i-1][j] + \delta,\ OPT[i][j-1] + \delta\big) & \text{otherwise} \end{cases}$$

The optimal cost is $OPT[m][n]$; the alignment strings are recovered by tracing back through the full table from (m, n) to $(0, 0)$.

Complexity: Time $O(mn)$, Space $O(mn)$ — the whole table must stay in memory to support traceback.

4.2 Memory-Efficient Divide and Conquer (`efficient.py`)

This implementation uses **Hirschberg's algorithm**, which reaches the same optimal cost in the same $O(mn)$ time but uses only $O(m + n)$ space, by never materializing the full table.

Linear-space cost rows. `last_row_costs(X, Y)` computes only the final row of the DP table, keeping just two rows (previous/current) in memory at a time.

Divide step. Split X at its midpoint into X_{left} and X_{right}; compute the forward cost row for X_{left} vs. Y, and the backward cost row for reversed X_{right} vs. reversed Y. The column j^* minimizing the sum of forward and backward cost marks where the optimal path crosses the midpoint row.

Conquer step. Recurse on $(X_{\text{left}}, Y[0{:}j^*])$ and $(X_{\text{right}}, Y[j^*{:}])$ and concatenate the results. The recursion bottoms out at a small full DP + traceback (`small_dp`) once either sequence has length ≤ 1.

Complexity: Time $O(mn)$ (recomputed cost rows at each recursion level still sum to $O(mn)$ overall), Space $O(m + n)$.

5 Correctness Verification

`compare.py` is a randomized differential test: it generates random DNA strings (length 1–80), runs both algorithms, and checks they report the *same optimal cost*. It does not require byte-identical alignment strings, since ties at the optimal cost can be broken differently by the two algorithms. Over 200 trials: **identical optimal cost 200/200 (100%)**, byte-identical alignment strings 48/200 (24%) — both expected outcomes, since cost-optimality is guaranteed but the alignment itself need not be unique.

6 Benchmark Methodology

`benchmark.py` sweeps problem size $m + n \in \{128, 256, 512, 1024, 2048, 3072, 4096\}$ and measures CPU time (`time.perf_counter()`) and peak memory via Python's `tracemalloc` (actual bytes allocated, a cleaner signal than an RSS delta). Results are written to `results.csv`.

7 Results

$m+n$	Basic (ms)	Efficient (ms)	Basic peak (KB)	Efficient peak (KB)	Mem. saved	Time cost
128	4.0	6.6	147.4	14.1	90.4%	1.67×
256	13.9	23.5	588.7	19.0	96.8%	1.69×
512	47.7	89.1	2,337.4	37.9	98.4%	1.87×
1024	300.9	419.4	9,291.5	76.2	99.2%	1.39×
2048	1,455.9	1,914.2	37,025.7	150.8	99.6%	1.31×
3072	3,471.9	4,702.0	83,189.3	224.7	99.7%	1.35×
4096	6,422.1	8,697.2	147,785.5	298.5	**99.8%**	1.35×

Time. Both algorithms scale as $O(mn)$ – time roughly quadruples each time $m+n$ doubles. Hirschberg's is consistently **1.3–1.9× slower**, since the divide step recomputes forward/backward cost rows at every recursion level; this is the expected cost of trading memory for arithmetic.

Memory. The basic DP table grows quadratically; Hirschberg's $O(m+n)$ rows grow linearly. The savings widen with input size, from 90.4% at $m+n=128$ to **99.8%** at $m+n=4096$ (147.8 MB down to 0.3 MB).

Takeaway. Hirschberg's algorithm is the clear choice once sequence length makes the quadratic-memory basic DP impractical, at the cost of a modest, bounded runtime penalty.

8 Project Structure

```
Sequence-Alignment/
|-- basic.py Basic Dynamic Programming implementation
|-- efficient.py Hirschberg's memory-efficient implementation
|-- basic.sh Runs the basic implementation
|-- efficient.sh Runs the efficient implementation
|-- compare.py Differential test: both algorithms vs. the DP optimum
|-- benchmark.py Time/memory sweep across input sizes -> results.csv
|-- results.csv Measured output of benchmark.py
|-- sample_input.txt Small example in the input format described above
|-- Summary.pages Project report
```

9 Running the Project

Both commands below run on `sample_input.txt` as-is and print an optimal alignment cost of **660**.

```
# Basic algorithm
./basic.sh sample_input.txt output.txt
python3 basic.py sample_input.txt output.txt

# Memory-efficient algorithm
./efficient.sh sample_input.txt output.txt
python3 efficient.py sample_input.txt output.txt

# Correctness check and benchmark
python3 compare.py
python3 benchmark.py
```

Each run writes an output file containing, in order: the minimum alignment cost, the two aligned sequences, the execution time, and the peak memory usage.

10 Requirements

Python 3.x and `psutil` (`pip install psutil`), used by `basic.py` and `efficient.py` to report peak memory. The alignment algorithms themselves use only the Python standard library; `benchmark.py` and `compare.py` need nothing beyond that.

11 Applications

DNA sequence alignment, computational biology, bioinformatics, genome comparison, evolutionary analysis.

12 References

- Needleman, S. B., & Wunsch, C. D. (1970). *A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of Two Proteins.* Journal of Molecular Biology, 48(3), 443–453.
- Hirschberg, D. S. (1975). *A Linear Space Algorithm for Computing Maximal Common Subsequences.* Communications of the ACM, 18(6), 341–343.